Observing and Controlling Features in Vision-Language-Action Models

Hugo Buurmeijer[1], Carmen Amo Alonso[1], Aiden Swann[2], and Marco Pavone[1,3]

arXiv:2603.05487v2 [cs.RO] 21 Sep 2026

***Abstract*— Vision-Language-Action models (VLAs) have shown remarkable progress towards embodied intelligence. While their architecture partially resembles that of Large Language Models (LLMs), VLAs exhibit higher complexity due to their multi-modal inputs/outputs and often hybrid nature of transformer and diffusion heads. This is part of the reason why insights from mechanistic interpretability in LLMs, which explain how the internal model representations relate to their output behavior, do not trivially transfer to VLA counterparts. In this work, we investigate whether VLA internal representations support lightweight behavioral steering without retraining. Across four frontier VLA models, linear *observers* recover state- and action-relevant information in both autoregressive and transformer–flow-matching architectures, and provide robust directions to causally alter VLA outputs. Building on this, we introduce a *controller* that minimally modifies representations to place observer predictions within prescribed target intervals. Closed-loop experiments with $\pi_{0.5}$ in the LIBERO simulator and on DROID hardware demonstrate improved constraint satisfaction while retaining task performance, with only approximately 1% inference overhead. Together, these experiments show that lightweight linear interventions can reliably steer VLA behavior while preserving closed-loop capabilities, enabling alignment with user preferences and task requirements without fine-tuning.**

I. Introduction

Vision-Language-Action models (VLAs) represent a significant step towards embodied intelligence. By processing multi-modal inputs consisting of images, language instructions, and proprioceptive states, VLAs combine perception, reasoning, and action generation in a single model. Similar to other disciplines where generative models have unlocked unprecedented generalization, VLAs enable robots to interpret natural language commands in rich visual contexts and execute complex behaviors, exhibiting improved generalization across tasks and environments [1]. However, VLAs suffer from similar limitations to other generative models: their behavior can be unpredictable, difficult to correct in real time, or misaligned with user preferences and safety requirements [2].

One promising approach to overcoming these limitations is the ability to steer generative models toward desired behaviors while suppressing undesirable ones. In the context of Large Language Models (LLMs), intervening internal model representations to influence outputs has become a prominent research direction, referred to as "activation steering" [3]. Achieving a similar level of control is arguably even more critical for VLAs, where steering directly affects physical behavior in the real world. Furthermore, since VLAs often share architectural components with LLMs, (most notably transformer-based representations [4]), methods developed for LLMs could in theory transfer to VLAs without significant overhead. However, VLAs operate over multiple modalities, including proprioceptive inputs, and must function in closed-loop interaction with the physical world where compounding errors are notorious, thereby posing challenges absent in LLMs. Additionally, robotic tasks such as manipulation naturally demand smooth, continuous actions, which has driven frontier architectures toward hybrid transformer-diffusion designs [5], further complicating the transfer of interpretability insights from LLMs to VLAs.

To close this gap, several works have explored porting concepts from mechanistic interpretability in LLMs over to the domain of VLAs [6], [7], [8], [9]. For instance, [6] overwrites VLA model activations at inference time to modulate basic motion features such as speed and direction. Overall, the effectiveness of existing methods is still limited. Crucially, while the focus of activation steering methods in LLMs is to control the behavior of the model while preserving its original merits, such as natural-sounding generations, in the field of VLAs interventions that preserve natural behaviors have not been studied. Moreover, the effect of architectural design choices, such as purely transformer-based versus hybrid transformer-diffusion architectures, on the interpretability and steerability of VLA representations remains unexplored.

In this paper, we address these challenges by formalizing concepts that are often implicit in work on steering LLMs, with the goal of making them applicable to VLAs. We introduce a unified framework for analyzing and influencing VLA behavior through their internal representations (see Figure 1), and use them to systematically compare common VLA architectures. We find linearly predictive state and action information in both autoregressive and transformer–flow-matching architectures, with performance varying across models, datasets, and features. Building on these findings, we develop lightweight observers and representation controllers, and demonstrate behavioral steering while retaining task performance in closed-loop simulation and hardware experiments.

Concretely, our **contributions** are as follows:

- We formalize the concepts of *feature-observability* and *feature-controllability* for VLAs, inspired by classical notions of observability and controllability, providing a framework to analyze and influence behavior through internal representations. (Section III-B)
- We systematically evaluate linear observability of OpenVLA, π_0, π_0-FAST, and $\pi_{0.5}$ across LIBERO, DROID, and BridgeData V2 datasets. We identify predictive information in two architecture families; states are generally

[1]Department of Aeronautics and Astronautics, Stanford University, USA. [2]Department of Mechanical Engineering, Stanford University, USA. [3]NVIDIA Research, USA.

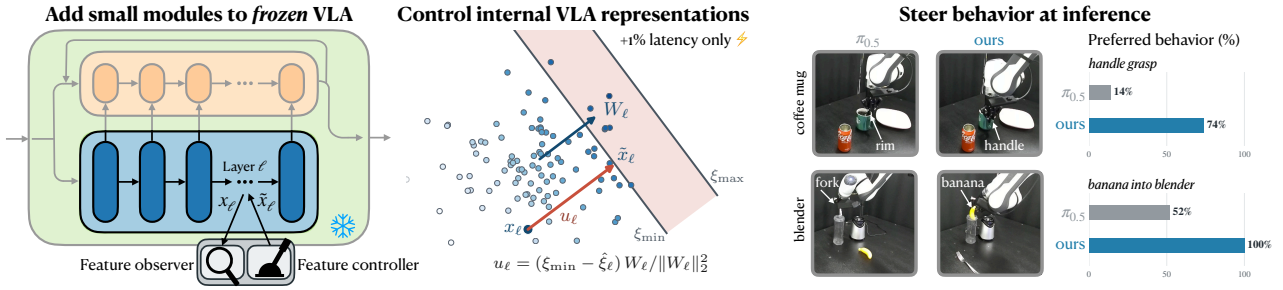

Fig. 1: We present the combination of *feature-observability* and *feature-controllability*: a framework to observe and control robot behavior through accessing and modifying internal VLA representations. A linear observer estimates features from the Transformer's internal representations, while a minimal additive controller steers these representations to align outputs with desired constraints. We demonstrate effective policy steering without fine-tuning in both simulation and on hardware.

more decodable than actions, with $\pi_{0.5}$ achieving the highest mean R^2 across continuous features on both LIBERO and DROID datasets. (Sections IV-A, V-A)

- We propose a minimal linear controller that leverages linear observability, and show that steering along observed directions smoothly affects the model outputs. (Sections IV-B, V-B)
- We present an online algorithm that integrates the observer and controller into closed-loop deployment without retraining. Simulation and hardware experiments on $\pi_{0.5}$ demonstrate the effectiveness of the proposed representation intervention. (Sections IV-C, V-C)

II. Related Work

A. Mechanistic Interpretability in VLAs

The intersection of mechanistic interpretability and robotics, specifically VLAs, remains significantly underexplored, though a number of recent works have investigated these tools in the context of end-to-end robot control. Notably, steering vectors can be extracted from the feed-forward layers of the Transformer backbone to causally affect model behaviors such as speed and height [6]. More abstract concepts, such as spatial relationships between objects and movement targets, were extracted from OpenVLA representations in [8] using linear probes, while [10] probe for recoverable state-transition structure. Such tools have also been used to improve downstream performance, e.g. by mechanistically identifying and fine-tuning task-relevant attention heads [7].

A complementary line of work forgoes pre-specified concepts in favor of unsupervised feature discovery: concurrent work of [9] trains Sparse Autoencoders on $\pi_{0.5}$ residual streams, proposes activation statistics to classify generality of features, and validates them via additive and ablative steering. Our contributions are complementary: because we target features specified a priori, observability and controllability are measured against a known ground truth rather than against post-hoc inspection of top-activating episodes.

B. Activation Steering in LLMs

A growing body of work has shown that semantic and behavioral features in LLMs are often linearly represented in their activation spaces, a phenomenon commonly referred to as the *linear separability hypothesis* [11]. It has been widely observed that simple linear classifiers can reliably extract attributes such as sentiment, topic, or personality traits from intermediate transformer layers [12], [13], [14], suggesting that these representations encode disentangled, behaviorally meaningful features. Building on this insight, *activation steering* methods modify internal activations at inference time to influence model outputs without fine-tuning [15], [16]. Techniques such as activation addition [3], [17] have been applied to guide generation toward desired attributes, mitigate harmful behaviors, and improve safety and alignment. Crucially, these methods emphasize preserving the naturalness and coherence of generated text by applying low-rank or minimal perturbations to the model's internal states.

III. Problem Statement

In this section, we present the problem of how the internal representations in VLAs can be controlled to align their outputs with user preferences. To that end, we describe the internal structure of two common VLA architectures, exemplified by OpenVLA [18] and $\pi_{0.5}$ [1]. We then introduce two concepts that enable the formalization of the problem in mathematical terms.

A. Vision-Language-Action Models (VLAs)

The goal is to formalize the study of internal representations in VLAs. Here we present two different types of VLA architectures (Figure 2): (a) autoregressive transformer-based, and (b) transformer-flow-matching hybrids.

*a) **Transformer**-based **VLAs**:* This architecture consists of a Vision-Language-Model autoregressive transformer architecture, where visual observations (processed through vision encoders such as DINOv2 and SigLIP) and language instructions are encoded into a shared token space, and a language model backbone (e.g., Llama 2) predicts tokenized output actions autoregressively, which are then decoded into robot commands. Examples of this architecture are OpenVLA [18], π_0-FAST [19] and RT2 [20].

*b) **Transformer**-**Flow-Matching** hybrid **VLAs**:* This architecture consists of an autorregressive transformer architecture, where a pretrained Vision-Language Model (VLM) processes visual and language inputs, while a separate "action expert" transformer uses conditional flow matching

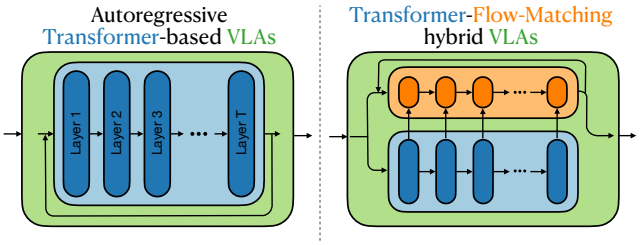

Fig. 2: Schematic representation of two VLA architectures. On the left, a *Transformer-based VLA* features an autoregressive Transformer architecture. On the right, a *Transformer-Flow-Matching hybrid VLA* is composed of a Transformer architecture and a Flow Matching block (referred to as the "action expert"), where the Flow Matching layers attend to their paired counterparts in the Transformer.

to generate continuous, high-frequency action trajectories. Examples of this architecture are π_0 [21] and $\pi_{0.5}$ [1].

We formalize the Transformer architecture [4] as a sequence of transformations (layers) acting on a latent representation of an input sequence. We denote the input by $w \in \mathcal{W}$, where $\mathcal{W}$ is the space of visual observations, language instructions, and proprioceptive states supplied to the model. An encoding map E, incorporating modality-specific encoders and projections, maps w to an initial hidden state $X_0 = E(w) \in \mathbb{R}^{N\times d}$, where N is the resulting number of token embeddings and d is their dimension. Then, each layer $\mathsf{T}_\ell : \mathbb{R}^{N\times d} \to \mathbb{R}^{N\times d}$ transforms the hidden state from the previous layer, yielding

$$X_{\ell+1} = \mathsf{T}_{\ell+1}(X_\ell), \qquad \ell = 0, \dots, L-1, \tag{1}$$

where $X_\ell \in \mathbb{R}^{N\times d}$ is the full hidden state after the ℓ-th layer.

For the subsequent analysis, we summarize the full token matrix X_ℓ by its mean-pooled representation

$$x_\ell := \frac{1}{N}\sum_{i=1}^{N} (X_\ell)_{i,:}^\top \in \mathbb{R}^d, \tag{2}$$

where $(X_\ell)_{i,:}$ denotes the i-th row of X_ℓ. As is common for LLM representations, we assume mean pooling preserves most feature-relevant information. Whether that information is present in VLAs is what we study.

This Transformer architecture is then embedded into a VLA model, where the action $a \in \mathcal{A} \subseteq \mathbb{R}^m$ is computed using the internal representations, i.e., $a = \phi(X_1, \dots, X_L)$. This map ϕ depends on the choice of architecture: for the transformer-based architecture ϕ depends only on the final layer representation, X_L, while for the transformer-flow-matching hybrid architecture ϕ includes the flow matching process that can be conditioned on intermediate layer representations $X_1, \dots, X_L$. The pooled vectors x_ℓ serve as summaries for our analysis; we do not assume that they alone determine the action.

When deployed in the real world, each sampled action a is executed in the environment, producing a new input and, over repeated interaction, a closed-loop trajectory $\tau \in \mathcal{T} := (\mathcal{W}\times\mathcal{A})^*$ of the form $\tau = (\tau_0, \tau_1, \dots)$, where $\tau_t := (w_t, a_t)$ is the t-th transition. The $(\cdot)^*$ operator captures finite, variable-length trajectories.

B. Feature Observability and Controllability

A *feature* is a measurable attribute of the model's output or the robot's behavior during execution, such as an action displacement, grasp modality, or the identity of a manipulated object. Formally, a feature is a map $\xi : \mathcal{T} \to \mathbb{R}^n$ assigning to each trajectory a value $\xi(\tau)$; this may be a direct readout of the trajectory, but can also be a label provided by a human annotator inspecting the rollout.

In this paper, we explore how the structure of the representation space in VLAs can be leveraged to (i) observe features, and (ii) steer features into a desired region by controlling their representations. To that end, we introduce the concepts of *feature-observability* and *feature-controllability*. Both are properties of a feature relative to a collection of inputs to the VLA: the requirement is that a *single* map works for every input in that collection, so the property is a statement about the representation space at a layer rather than about an individual rollout.

Definition III.1 (Feature-Observability)**.** A feature ξ is *observable at layer* ℓ if there exists a single map $f_\ell \in \mathcal{F}$ such that $f_\ell(x_\ell) = \xi(\tau)$ for every input. The map f_ℓ is referred to as the *observer*.

Definition III.2 (Feature-Controllability)**.** Given a target set $\mathcal{C} \subseteq \mathbb{R}^n$, a feature ξ is *controllable to* $\mathcal{C}$ *at layer* ℓ if there exists a single map $g_\ell \in \mathcal{G}$ such that, for every input, setting $X_\ell = g_\ell(\mathsf{T}_\ell(X_{\ell-1}))$ yields a trajectory $\tilde{\tau}$ with $\xi(\tilde{\tau}) \in \mathcal{C}$. The map g_ℓ is referred to as the *controller*.

Remark III.3. The definitions above admit a broad range of features. One can distinguish (a) output-level features, assessed at a specific point in the trajectory, $\xi(\tau_t)$; and (b) trajectory-level features, assessed on the full trajectory, $\xi(\tau)$. Consequently a feature need not be observable or controllable at every time step: an eventual grasp modality may not yet be distinguishable early in execution, while a choice already realized cannot be undone by later interventions.

In the general case, the set of observers and controllers considered are $\mathcal{F} \subseteq \{f : \mathbb{R}^d \to \mathbb{R}^n\}$ and $\mathcal{G} \subseteq \{g : \mathbb{R}^{N\times d} \to \mathbb{R}^{N\times d}\}$. In practice the observer and controller are usually learned rather than prescribed, so $\mathcal{F}$ and $\mathcal{G}$ are restricted to a specific structure (linear maps, one-layer MLPs, etc.) and fitted on a collection of inputs drawn from rollouts together with their representations and feature values.

We state both notions exactly, as the idealized versions make the structure clearest. Empirically neither holds exactly: we report the accuracy with which a fitted observer recovers $\xi(\tau)$, and the rate at which a controller lands in $\mathcal{C}$, and read these as graded evidence for the corresponding property.

Given the definitions above, the open question is which layers admit observers and controllers for features of interest, and how small the classes $\mathcal{F}$ and $\mathcal{G}$ can be while still doing so. In what follows we propose that linear choices suffice: linear observers recover features of interest from x_ℓ, and linear, additive controllers steer the resulting behavior toward specified targets $\mathcal{C}$ during execution.

IV. Approach

In this section we instantiate the two introduced concepts, *feature-observability* and *feature-controllability*, with concrete choices of $\mathcal{F}$ and $\mathcal{G}$. We propose a linear *observer*, which extracts the value of an interpretable feature from the model's internal state, and an additive, linear *controller*, which intervenes on that state to steer the resulting behavior toward a target set. Both are cheap enough to apply online during execution, requiring a single matrix multiply per forward pass. We evaluate how well they hold empirically in Section V.

A. Designing the Feature Observer

We design the observer map at layer ℓ, $f_\ell(\cdot)$, to be a linear function, i.e. we restrict $\mathcal{F}$ to be the set of linear maps from $\mathbb{R}^d$ to $\mathbb{R}^n$. This is, given a representation $x_\ell \in \mathbb{R}^d$, the feature of interest is predicted as $\hat{\xi} = f_\ell(x_\ell)$, with

$$f_\ell(x) := W_\ell^\top x + b_\ell, \tag{3}$$

where $W_\ell \in \mathbb{R}^{d\times n}$ and $b_\ell \in \mathbb{R}^n$ are learned from data. Specifically, we leverage pairs of inputs (w containing images, prompt and potentially robot states) to a feature of interest (denoted as $\xi \in \mathbb{R}^n$). Given a set of input-feature pairs $\{w^{(i)}, \xi^{(i)}\}_{i=1}^K$, a set of activation-feature pairs $\{x_\ell^{(i)}, \xi^{(i)}\}_{i=1}^K$ can be collected at each layer ℓ by simply propagating each input sequence $w^{(i)}$ up until layer ℓ through the transformer architecture, whose output is $x_\ell^{(i)}$. Then, W_ℓ and b_ℓ are learned by minimizing a task-appropriate loss over $\{x_\ell^{(i)}, \xi^{(i)}\}_{i=1}^K$:

$$W_\ell, b_\ell = \arg\min_{W,\, b} \sum_{i=1}^{K} \mathcal{L}\big(W^\top x_\ell^{(i)} + b, \xi^{(i)}\big), \tag{4}$$

where $\mathcal{L}$ is the mean squared error loss for continuous features ($\xi \in \mathbb{R}^n$) and the cross-entropy loss for discrete features ($\xi \in \{0,1\}^n$). Algorithm 1 summarizes the steps to train the observer at layer ℓ, $f_\ell(\cdot)$.

The choice of a linear map is motivated by the linear separability hypothesis, widely explored in LLMs [11]. We note that the presented method holds for generalized versions of the linear observer, such as $f_\ell(x) = \nu(W_\ell^\top x + b_\ell)$, where ν is some known monotonic nonlinearity of choice.

Algorithm 1 Learning linear observer f_ℓ (Eq. (3)) at layer ℓ

Input: Labeled dataset $\{(w^{(i)}, \xi^{(i)})\}_{i=1}^K$
Output: $W_\ell \in \mathbb{R}^{d\times n}$ and $b_\ell \in \mathbb{R}^n$
for $i = 1, \dots, K$ **do** ▷ In parallel for all $i = 1, \dots, K$
  $x_0^{(i)} \leftarrow E(w^{(i)})$ ▷ Compute initial embedding
  for $\ell = 0, \dots, L-1$ **do** ▷ Until layer L
    $x_{\ell+1}^{(i)} \leftarrow$ Eq. (1) ▷ Get layer ℓ representation
  end for
end for
$(W_\ell, b_\ell) \leftarrow$ Eq. (4) ▷ Train linear observer at layer ℓ
return W_ℓ, b_ℓ

B. Designing the Feature Controller

We design the controller map at layer ℓ to act additively on the full token representation, broadcasting a single perturbation $u_\ell \in \mathbb{R}^d$ to every token. That is, we restrict $\mathcal{G}$ to maps of the form

$$g_\ell(X) := X + \mathbf{1}_N\, u_\ell(x)^\top, \qquad x = \tfrac{1}{N} X^\top \mathbf{1}_N, \tag{5}$$

where $\mathbf{1}_N \in \mathbb{R}^N$ is the all-ones vector and the perturbation depends on X only through its mean-pooled summary x. This shifts the pooled representation by exactly $u_\ell := u_\ell(x_\ell)$.

Broadcasting is not an arbitrary choice: it is the least-norm way to realize a prescribed shift of the pooled summary.

Proposition IV.1 (Minimum-norm realization of a pooled perturbation)**.** *For a prescribed pooled perturbation $u \in \mathbb{R}^d$, the broadcast perturbation $\Delta X^\star = \mathbf{1}_N u^\top$ is the unique minimizer of $\|\Delta X\|_F^2$ over $\Delta X \in \mathbb{R}^{N\times d}$ satisfying $\frac{1}{N}\Delta X^\top \mathbf{1}_N = u$. Its squared Frobenius norm is $N\|u\|_2^2$.*

Proof. Write $\delta_j = \Delta X_{j,:}^\top$, so that $\sum_{j=1}^N \delta_j = Nu$. Expanding about u,

$$\|\Delta X\|_F^2 = \sum_{j=1}^N \|\delta_j\|_2^2 = N\|u\|_2^2 + 2u^\top \sum_{j=1}^N (\delta_j - u) + \sum_{j=1}^N \|\delta_j - u\|_2^2,$$

and the middle term vanishes by the constraint. Hence $\|\Delta X\|_F^2 \geq N\|u\|_2^2$, with equality iff $\delta_j = u$ for all j. □

Following prior work in LLMs [12], we compute u_ℓ as the solution to the optimization problem:

$$u_\ell = \arg\min_{u\in\mathbb{R}^d} \quad \|u\|_2^2 \tag{6a}$$
$$s.t. \quad f_\ell(x_\ell + u) \in \mathcal{C}, \tag{6b}$$

where $\mathcal{C} \subseteq \mathbb{R}^n$ is the desired set of feature values, and $f_\ell(\cdot)$ is the observer map at layer ℓ. This optimization finds the smallest pooled perturbation that places the observer output in $\mathcal{C}$, and by Proposition IV.1 it is also the smallest intervention on the full token representation, since $\|\Delta X\|_F^2 = N\|u\|_2^2$.

In general, the constraint in (6b) may be nonconvex, depending on the form of f_ℓ and the geometry of $\mathcal{C}$. To obtain a tractable solution, we use the linear observer in (3), and consider a scalar feature with target interval $\mathcal{C} = [\xi_{\min}, \xi_{\max}] \subseteq \mathbb{R}$, where $\xi_{\min} \leq \xi_{\max}$. For $W_\ell \neq 0$, the solution to (6) has the following closed form [12, Theorem 4.1]:

$$u_\ell = (\xi_{\max} - \hat{\xi}_\ell)\frac{W_\ell}{\|W_\ell\|_2^2}, \qquad \text{if } \hat{\xi}_\ell > \xi_{\max}, \tag{7a}$$
$$u_\ell = (\xi_{\min} - \hat{\xi}_\ell)\frac{W_\ell}{\|W_\ell\|_2^2}, \qquad \text{if } \hat{\xi}_\ell < \xi_{\min}, \tag{7b}$$
$$u_\ell = 0, \qquad \text{otherwise}, \tag{7c}$$

where $\hat{\xi}_\ell = f_\ell(x_\ell) = W_\ell^\top x_\ell + b_\ell$ is the layer ℓ observation.

Observability and controllability are independent properties [22], but methods from the LLM literature show that steering using a learned readout of the feature [12], [13] outperform those injecting directions without one [3]. Therefore, our proposed controller relies on the observer.

Fig. 3: Linear observability of robot states and actions inside four vision-language-action models. Each row is one model evaluated on one dataset. For every quantity a linear probe is trained per layer on pooled Transformer activations, and scored on a held-out 10% split. Heatmaps report R^2 for the six continuous states and actions, taking the best of the probed layers per quantity. Dot plots report accuracy on binary quantities, gripper state g and gripper command Δg, where 50% is chance.

C. Observing and Controlling Features in VLAs

We illustrate how to leverage the proposed feature observer and feature controller in practice to align VLA outputs with user preferences. Given

- a *feature observer*, $f_\ell(\cdot)$, with linear structure as defined in Equation (3), where linear parameters W_ℓ and b_ℓ are computed before inference time via Algorithm 1; and
- a *feature controller*, $g_\ell(\cdot)$, with linear structure as defined in Equation (5), where the additive u_ℓ is computed according to Equation (7),

the goal is to integrate them into the inference time computations at layer ℓ to achieve precise representation control and downstream alignment of the VLA output.

Let us consider two set of layers: (i) $\Lambda_O \subseteq [0, \dots, L]$ where feature-observability is desired, and (ii) $\Lambda_C \subseteq [0, \dots, L]$ where feature-controllability is desired. We note that, given that the proposed controller requires observations, $\Lambda_C \subseteq \Lambda_O$. The inference-time forward pass equipped with observer and controller for feature ξ is described by Algorithm 2.

The forward-pass presented in Algorithm 2 only differs from a standard un-observed and un-intervened Transformer forward-pass in steps 6-12. A crucial advantage of choosing a linear observer and a linear closed-form controller is that computations in step 7, 8 and 9 introduce minimal overhead, resulting in a negligible increase in extra runtime.

Algorithm 2 Inference forward-pass in a VLA-embedded Transformer architecture with ξ-observer and ξ-controller

1: **Input:** w, Λ_O, Λ_C, $W_{\ell\in\Lambda_O}$, $b_{\ell\in\Lambda_O}$, $\xi_{\min}$, $\xi_{\max}$
2: **Output:** $x_1, \dots, x_L$
3: $x_0 \leftarrow E(w)$ ▷ Compute initial embedding
4: **for** $\ell = 1, \dots, L$ **do** ▷ **Forward-pass through layers**
5: $x_\ell \leftarrow$ Eq. (1) ▷ Compute layer ℓ representation
6: **if** $\ell \in \Lambda_O$ **then** ▷ **Feature-observability**
7: $\xi_\ell \leftarrow$ Eq. (3) ▷ Compute observation
8: **if** $\ell \in \Lambda_C$ **then** ▷ **Feature-controllability**
9: $u_\ell \leftarrow$ Eq. (7) ▷ Compute control input
10: $x_\ell \leftarrow$ Eq. (5) ▷ Apply intervention
11: **end if**
12: **end if**
13: **end for**
14: **return** $x_1, \dots, x_L$

V. Experiments

In this section, we present the experiments that explore the concepts of feature-observability and feature-controllability in frontier VLA policies: the transformer-based OpenVLA and π_0-FAST and the transformer-flow-matching hybrid models π_0 and $\pi_{0.5}$. We first compare how accurately robot states and actions can be observed in representation space using a linear observer across these architectures. Then, we show that targeted perturbations in the representation space smoothly affect VLA outputs and that precise representation control is achieved. Lastly, we study how minimal control interventions to the representation space lead to effective VLA steering in *closed-loop* deployment on hardware.

A. Observing Features in VLA Representations

We assess whether robot-relevant features can be predicted using the linear observer defined in Equation (3), and how prediction performance varies across models and datasets. We evaluate π_0, π_0-FAST, and $\pi_{0.5}$ on LIBERO [23] and DROID [24], and OpenVLA [18] on BridgeData V2 [25]. We focus on robot states and actions, which are available in these datasets, and fundamental robot quantities. Following Remark III.3, these are output-level features, while trajectory-level features are evaluated in Section V-C.

The state features are $s = (x, y, z, \phi, \theta, \psi, g)$, comprising Cartesian end-effector position, roll, pitch, yaw, and normalized gripper aperture. Action features comprise translational and rotational commands $(\Delta x, \Delta y, \Delta z, \Delta\phi, \Delta\theta, \Delta\psi)$ and the gripper command Δg, following each dataset's conventions. For chunked actions, labels are averaged over the chunk; OpenVLA uses single-step action labels. Continuous quantities are predicted using regression probes, while gripper state and command are predicted using binary classifiers.

For each model–dataset pair, we construct input–feature pairs $\{(w^{(i)}, \xi^{(i)})\}_{i=1}^K$, where, for example, $\xi = \Delta x \in \mathbb{R}$. Inputs contain scene images, task instructions, and proprioceptive measurements as supported by each model; OpenVLA receives a single image and no proprioceptive measurements. We pass these inputs through the frozen models and collect

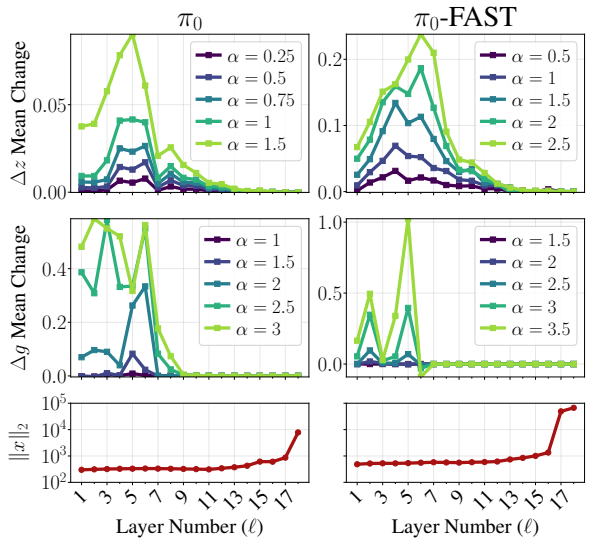

Fig. 4: Output sensitivity to observer-aligned representation interventions in the LIBERO checkpoints of π_0 (left) and π_0-FAST (right). Top and middle: mean changes in vertical displacement Δz and gripper command Δg relative to the unmodified model, as a function of intervention layer, for different strengths α. Bottom: representation L_2-norm across layers, on a logarithmic scale.

pooled representations at each layer, yielding activation–feature pairs $\{(x_\ell^{(i)}, \xi^{(i)})\}_{i=1}^K$. For all models, we probe the VLM prefix, excluding action-expert activations in hybrid architectures. For autoregressive models, activations are collected during the first forward pass, before action-token generation. Observers are trained separately for each feature and layer using Algorithm 1, and evaluated on a held-out 10% split. Figure 3 reports the highest score across layers.

Figure 3 shows that all evaluated model–dataset pairs contain linearly predictive information about robot states and actions. State features are recovered more accurately than action features throughout: Cartesian state probes achieve $R^2 = 0.81$–1.00 and orientation probes $R^2 = 0.53$–0.99, while the corresponding action probes fall well below this. Similarly, gripper-state classification is close to perfect, while gripper-command accuracy shows a larger spread. We also note that $\pi_{0.5}$ on average has the strongest decodability of continuous features, but not for binary (gripper) features.

Action decodability is markedly lower on the real-world datasets while state decodability is not, dropping by roughly a third from LIBERO to DROID with state scores essentially unchanged. Since the gap is specific to actions, one explanation is that the LIBERO policies encode a comparatively rigid state-to-action mapping fit to scripted demonstrations, whereas policies trained on diverse teleoperation data retain more distributed and multimodal action representations. Real-world action targets are also intrinsically noisier, so lower R^2 need not by itself imply less linear structure.

Overall, linear predictability of state and action features is present in autoregressive and transformer–flow-matching architectures alike, with the simulated–real gap in action decodability larger than any gap between architectures.

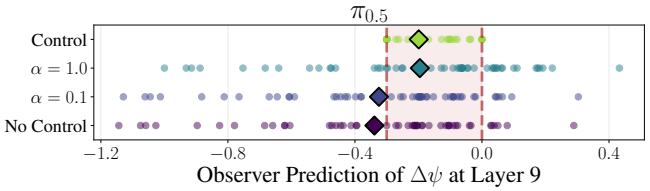

Fig. 5: Representations projected into the classifier image space at layer 9 of $\pi_{0.5}$, under different interventions. Individual samples are shown with means indicated by solid diamonds. The controller constrains the representations to lie within the target bounds $[\xi_{\min}, \xi_{\max}]$, while baseline interventions and un-intervened representations fall outside.

B. Controlling Features Through Representation Interventions

Feature predictability alone does not establish that a representation can be used to steer model behavior. We therefore examine two requirements for representation control: whether observer-aligned perturbations produce graded changes in the generated actions, and whether the controller can place individual observer predictions within a prescribed target.

Output sensitivity to representation interventions. We compare the LIBERO checkpoints of π_0 and π_0-FAST to examine flow-matching and autoregressive action generation within the same model family. For each scalar action feature, we perturb the representation along its linear observer direction, $x_\ell \leftarrow x_\ell + \alpha W_\ell$, where α controls intervention strength. Only one layer is perturbed per generation. We sweep the layer and intervention strength, measuring the mean change in the corresponding output feature relative to the unmodified model across 50 LIBERO episodes.

Across the six continuous action dimensions, increasing α produces graded, generally ordered changes in the mean output over the tested intervention strengths. Figure 4 illustrates this behavior for the vertical displacement command Δz. The trend appears in both models, indicating that observer-aligned interventions can modulate continuous actions under both generation mechanisms. Similar continuous-action responses are observed for the LIBERO and DROID checkpoints of $\pi_{0.5}$, which we use in subsequent closed-loop experiments. The binary gripper response is less regular, while still showing an ordering with increasing α. Lower sensitivity in later layers may partly be attributed to increasing representation scales, which can reduce the relative effect of a fixed perturbation.

Control of observer predictions. We next evaluate the minimal controller in Equation (7) in the output space of the scalar affine observer from Equation (3). A fixed additive perturbation shifts all observer predictions by the same amount, preserving their spread. Consequently, it can move the mean prediction into the target interval while leaving individual samples outside, as Figure 5 illustrates. The proposed controller instead adapts the intervention to each representation, satisfying $f_\ell(\widetilde{x}_\ell) \in [\xi_{\min}, \xi_{\max}]$ by construction of the constraint in Equation (6). Figure 5 demonstrates this empirically. This guarantee concerns the observer output; behavioral constraint satisfaction during closed-loop execution is evaluated in the next section.

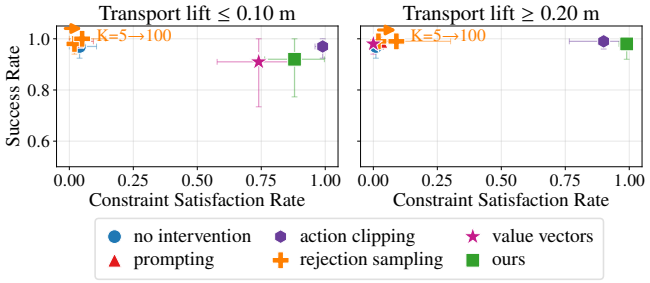

Fig. 6: Task success rate vs. constraint satisfaction in LIBERO-Spatial across 100 rollouts per method (10 tasks × 10). The lift during the transport phase is constrained above/below. Whiskers show ±1 standard deviation across tasks.

C. Observing and Controlling Features in Closed-Loop

The preceding experiments examined state and action observability and the effects of representation interventions on model outputs. We now evaluate whether these interventions steer executed behavior during closed-loop interaction, including grasp modalities and object choices, while preserving task performance. We use frozen $\pi_{0.5}$ checkpoints [1] in LIBERO simulation [23] and on the DROID hardware platform [24], with policy inference on a single NVIDIA RTX 5090 GPU.

Evaluation and baselines. We consider transport-lift constraints, preferred grasp modalities, and task-specific object-selection constraints. The three simulation experiments comprise 100 rollouts each, while each of the three hardware tasks uses 50 rollouts per method. Constraint satisfaction is the fraction of rollouts for which $\xi(\tau) \in \mathcal{C}$; for categorical features, this is the frequency of the preferred grasp or designated safe object. Task success is evaluated separately.

The main baselines are the unmodified policy, prompt interventions, and value-vector steering [6]. For lift constraints, we additionally evaluate two output-side baselines: rejection sampling and action clipping. Rejection sampling draws up to K action chunks at each replan, selecting a satisfying candidate when available and otherwise executing the least-violating candidate. Action clipping directly modifies the commanded motion, using a safety margin to account for discrepancies between commanded and executed trajectories. The 50 base policy rollouts are re-used to fit the observers.

Constraining transport motion. We impose either an upper bound of 0.10 m or a lower bound of 0.20 m on the vertical lift during the task's transport phase. Constraints apply over this phase, and task success requires completion of the original LIBERO task. Figure 6 shows that representation control substantially increases constraint satisfaction while retaining high task success. Prompting, on the other hand, fails to meaningfully alter the policy's behavior. Increasing the rejection-sampling budget $K = 5$ to $K = 100$ leaves satisfaction below 10% for both bounds, while value vectors achieve approximately 74% for the upper bound and 0% for the lower bound. As expected for these geometric constraints, action clipping performs well. In our experiments, only the lower-bound constraint requires tuning: the limited vertical displacement achievable in a single control step necessitates a margin (of 4 cm) that activates corrective motion early enough to reach the required lift during transport.

Fig. 7: Examples of closed-loop hardware behaviors. Left: grasping a coffee-filled mug by the rim (undesired) or handle (preferred). Right: selecting a metal fork (unsafe) or banana (safe) for a blender. Arrows indicate desired behavior change.

Rejection sampling also incurs substantial computational overhead. Even with shared prefix computation and batched action-expert evaluations, median inference time increases by 16% at $K = 5$ and 583% at $K = 100$, compared with approximately 1% for our closed-form representation interventions (measured over 50 forward passes).

Aligning grasp modalities with user preferences. We next constrain how the robot grasps an object. In LIBERO, the task is to pick up a wine bottle, with a more stable, preferred grasp at the base rather than the top. As shown in Table I, the base policy selects the preferred modality in only 28% of rollouts. The better of two prompts emphasizing grasp location or approach direction raises this to 40%, while value-vector steering reduces it to 3%. Representation control makes the preferred modality dominant at 73%.

On hardware, the robot must pick up a coffee-filled mug by its handle; a rim grasp places a gripper finger in the liquid. The base policy grasps the handle in only 14% of rollouts. Prompting and value vectors increase this to 46% and 38%, respectively, while our method reaches 74%. All methods successfully grasp the mug, so the improvement concerns the executed grasp modality rather than grasp success. Figure 7 illustrates the preferred and undesired behaviors. In both tasks, representation control makes the preferred modality dominant, although some rollouts still violate the preference.

Enforcing task-specific semantic constraints. Finally, we evaluate object selection in two hardware tasks: choosing a banana rather than a metal fork for a blender, and a glass rather than a plastic bowl for an oven. These constraints concern the identity of the manipulated object. The base policy selects the designated safe object in 52% and 48% of rollouts, respectively (Table II). Prompting reaches 40% and 62%, while value vectors reach 64% and 94%. Our method selects the designated safe object in all 50 evaluated

TABLE I: Grasp modality distribution across 100 sampled initial conditions in the LIBERO-Goal scene (wine bottle) and across 50 rollouts on hardware (coffee mug).

	Wine bottle		Coffee mug	
	Base ↑	Top ↓	Handle ↑	Rim ↓
Base policy ($\pi_{0.5}$)	28%	72%	14%	86%
Prompting	40%	60%	46%	54%
Value vectors [6]	3%	97%	38%	62%
Ours	**73%**	**27%**	**74%**	**26%**

TABLE II: Object manipulation distribution across 50 rollouts for two semantic safety experiments. In each, the safe object should be selected and the unsafe object should be avoided.

	Blender		Oven (bowls)	
	Banana ↑	Fork ↓	Glass ↑	Plastic ↓
Base policy ($\pi_{0.5}$)	52%	48%	48%	52%
Prompting	40%	60%	62%	38%
Value vectors [6]	64%	36%	94%	6%
Ours	**100%**	**0%**	**100%**	**0%**

rollouts of each task. Here, task success means successfully manipulating the selected object, which all methods achieve; full completion of the blender or oven task is not evaluated. We emphasize that our method does not guarantee safety in these scenarios, and hence can be complementary to output-level filters with action-space guarantees.

VI. Conclusion and Outlook

This paper introduced a principled framework for understanding and controlling Vision-Language-Action models through their internal representations. We used this framework to study how architectural design choices affect the interpretability and steerability of VLA internal representations, and leveraged these insights to enable reliable inference-time steering. We found linearly predictive state and action information in both autoregressive and transformer–flow-matching VLAs, with performance varying across models, datasets, and features. Our lightweight observers and controllers enable behavioral steering without retraining, adding approximately 1% inference overhead. Closed-loop experiments in the LIBERO simulator and on DROID hardware demonstrate improved constraint satisfaction while retaining task performance. On hardware, preferred grasps increased by 60 percentage points, and the designated safe objects were selected in all 50 rollouts, beating both prompting and value vector baselines.

Limitations and Future Work. While our framework successfully demonstrates feature observability and controllability, several directions warrant further investigation. First, our current approach requires labeled data to train linear observers, which may not always be available in large-scale robotics datasets. Second, we focused primarily on the Transformer components of VLA architectures; extending our framework to the diffusion or flow-matching heads would enable end-to-end interpretability and control across hybrid architectures. Finally, establishing principled safety guarantees and bounds on the effects of representation-space interventions would be crucial for deploying these techniques in safety-critical robotic applications.

References

[1] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al., "$\pi_{0.5}$: A vision-language-action model with open-world generalization," *arXiv preprint 2504.16054*, 2025.

[2] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, et al., "Libero-plus: In-depth robustness analysis of vision-language-action models," *arXiv preprint 2510.13626*, 2025.

[3] A. M. Turner, L. Thiergart, D. Udell, G. Leech, U. Mini, and M. MacDiarmid, "Activation addition: Steering language models without optimization," *CoRR*, 2023.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[5] K. Kawaharazuka, J. Oh, J. Yamada, I. Posner, and Y. Zhu, "Vision-language-action models for robotics: A review towards real-world applications," *IEEE Access*, 2025.

[6] B. Häon, K. Stocking, I. Chuang, and C. Tomlin, "Mechanistic interpretability for steering vision-language-action models," in *Conference on Robot Learning*, 2025.

[7] C. Mitra, Y. Luo, R. Saravanan, D. Niu, A. Pai, J. Thomason, T. Darrell, A. Anwar, D. Ramanan, and R. Herzig, "Mechanistic finetuning of vision-language-action models via few-shot demonstrations," *arXiv preprint 2511.22697*, 2025.

[8] H. Lu, H. Li, P. S. Shahani, S. Herbers, and M. Scheutz, "Probing a vision-language-action model for symbolic states and integration into a cognitive architecture," *IEEE International Conference on AI and Data Analytics*, 2025.

[9] A. Swann, L. McGranahan, H. Buurmeijer, M. Kennedy III, and M. Schwager, "Sparse autoencoders reveal interpretable and steerable features in vla models," in *Conference on Robot Learning*, 2026.

[10] M. Molinari, L. Nevali, S. Navani, and O. G. Younis, "Emergent world representations in openvla," *arXiv preprint 2509.24559*, 2025.

[11] K. Park, Y. J. Choe, and V. Veitch, "The linear representation hypothesis and the geometry of large language models," in *International Conference on Machine Learning*, PMLR, 2024.

[12] E. Cheng and C. A. Alonso, "Liseco: Linear semantic control for language generation," *Transactions on Machine Learning Research*, 2026.

[13] P. Rodriguez, A. Blaas, M. Klein, L. Zappella, N. Apostoloff, X. Suau, et al., "Controlling language and diffusion models by transporting activations," in *The Thirteenth International Conference on Learning Representations*, 2025.

[14] R. Chen, A. Arditi, H. Sleight, O. Evans, and J. Lindsey, "Persona vectors: Monitoring and controlling character traits in language models," *arXiv preprint 2507.21509*, 2025.

[15] Z. Wu, A. Arora, Z. Wang, A. Geiger, D. Jurafsky, C. D. Manning, and C. Potts, "Reft: Representation finetuning for language models," *Advances in Neural Information Processing Systems*, vol. 37, pp. 63 908–63 962, 2024.

[16] J. Sun, S. Baskaran, Z. Wu, M. Sklar, C. Potts, and A. Geiger, "Hypersteer: Activation steering at scale with hypernetworks," *arXiv preprint 2506.03292*, 2025.

[17] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg, "Inference-time intervention: Eliciting truthful answers from a language model," *Advances in Neural Information Processing Systems*, 2023.

[18] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "Openvla: An open-source vision-language-action model," in *Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, 2025.

[19] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine, "Fast: Efficient action tokenization for vision-language-action models," *arXiv preprint 2501.09747*, 2025.

[20] B. Zitkovich et al., "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *Proceedings of The 7th Conference on Robot Learning*, J. Tan, M. Toussaint, and K. Darvish, Eds., ser. Proceedings of Machine Learning Research, vol. 229, PMLR, 2023, pp. 2165–2183.

[21] K. Black et al., "π_0: A vision-language-action flow model for general robot control," *arXiv preprint 2410.24164*, 2024.

[22] R. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, 1960.

[23] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, "Libero: Benchmarking knowledge transfer for lifelong robot learning," *Advances in Neural Information Processing Systems*, vol. 36, pp. 44 776–44 791, 2023.

[24] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, et al., "Droid: A large-scale in-the-wild robot manipulation dataset," *arXiv preprint 2403.12945*, 2024.

[25] H. Walke, K. Black, A. Lee, M. J. Kim, M. Du, C. Zheng, T. Zhao, P. Hansen-Estruch, Q. Vuong, A. He, V. Myers, K. Fang, C. Finn, and S. Levine, "Bridgedata v2: A dataset for robot learning at scale," in *Conference on Robot Learning*, 2023.